

PrimalityTest

May 18, 2026

0.1 Primality Test - Implementazione e Analisi di $n = 65$

In questo laboratorio e' richiesto di verificare il comportamento dell'algoritmo $\text{MCPrimalityTest}(n)$ su $n = 65$, verificare che $H(65)$ sia un sottogruppo di $\mathbb{Z}_{65}^*$ e di produrre un *MR-Testimone* banale e non banale del fatto che 65 sia un numero *composto*.

Inizio definendo il comportamento dell'algoritmo e richiamando alcuni concetti teorici alla base di questo. Il protocollo si basa su due concetti chiave:

1. **Piccolo Teorema di Fermat**, il quale afferma che, considerati p *primo* e $a \in \mathbb{Z}$ non divisibile per p , $a^{p-1} \equiv 1 \pmod{p}$
2. **Radici quadrate dell'unita'**, ossia che, considerato p *primo*, $x_+ = 1$ e $x_- = -1$ sono le uniche soluzioni dell'equazione $x^2 \equiv 1 \pmod{p}$

Inoltre si ha che, considerato un qualsiasi $n \geq 3$, input dell'algoritmo, puo' essere riscritto con q dispari, $s \geq 1$ come $n = q \cdot 2^s + 1$ e conseguentemente possiamo scomporre $a^{n-1} - 1$ in una sequenza di moltiplicazioni $(a^{q2^{s-1}} + 1)(a^{q2^{s-2}} + 1) \dots (a^q + 1)(a^q - 1) \equiv 0 \pmod{n}$. Avremo che, se n e' *primo*, uno dei fattori di questa scomposizione sara' necessariamente nullo, ossia che il fattore $(X \pm 1)$ sia multiplo di n . Da questo otteniamo che esisteranno solo due sequenza possibili (*quadrature*), della forma $1, \dots, 1$, oppure $\dots, -1, 1, \dots, 1$.

Infine definiamo cosa intendiamo per *MR-Testimone*. Se per qualche *base* a , l'algoritmo produce l'*output* **n composto**, a sara' un *MR-Testimone* per n e n sara' **certamente composto**. Questo accade se incontriamo una sequenza di *quadrature* diversa da quelle precedentemente dichiarate. Se per n *composto*, una *base* a produce l'*output* **n probabilmente primo**, tale a sara' un *MR-Bugiardo* per n .

Vengono definiti *banali* nel caso in cui a e n non siano coprimi, se $\text{MCD}(a, n) > 1$. Le basi estreme, $a = 1$, $a = n - 1$ saranno sempre *MR-Bugiardi* per n *composto*.

```

MCPrimalityTest(n)
1. s = 0
2. q = n - 1
3. while (q pari)
    s ← s + 1
    q ← q/2
4. campiona a uniformemente a caso in {2,3,...,n - 2}
5. calcola x = a^q (mod n)
6. if x ≠ ±1
    return n probabilmente primo
7. while s - 1 = 0
    x = x^2 (mod n)
    if x = -1 (mod n)
        return n probabilmente primo
    s ← s - 1
8. return n composto

```

Considerato $n = 65$, troviamo la sua scomposizione in forma $q \cdot 2^s + 1$ con $q = 1, s = 6$. Sappiamo che $|\mathbb{Z}_{65}^*| = 48$. $H_{65} = \{1, 8, 14, 18, 47, 51, 57, 64\}$ e' un sottogruppo proprio di $\mathbb{Z}_{65}^*$ costituito dalla coppia di *MR-Banali* $\{1, 64\}$, dalle due coppie di *MR-Bugiardi* $\{8, 57\}$ e $\{18, 47\}$ e dalla coppia di *MR-Testimoni* $\{14, 51\}$.

Questo insieme inganna il **Piccolo Teorema di Fermat**, in quanto per qualsiasi $e \in H_{65}, e^{64} \equiv 1 \pmod{p}$, invece l'algoritmo sopra presentato riesce almeno a smascherare i *MR-Testimoni* in quanto generano una sequenza invalida (non delle forme accettate $1, \dots, 1$, o $\dots, -1, 1, \dots, 1$).

Verifichiamo, per la base $a = 2$ il funzionamento dell'algoritmo:

```

1 :  $x \equiv 2^1 \pmod{65} = 2$ 
2 :  $2 \not\equiv \pm 1$ 
3 :  $6 - 1 \geq 0 \rightarrow x \equiv 2^2 \pmod{65} = 4$ 
4 :  $4 \not\equiv -1 \pmod{65}$ 
5 :  $5 - 1 \geq 0 \rightarrow x \equiv 4^2 \pmod{65} = 16$ 
6 :  $16 \not\equiv -1 \pmod{65}$ 
7 :  $4 - 1 \geq 0 \rightarrow x \equiv 16^2 \pmod{65} = 61$ 
8 :  $61 \not\equiv -1 \pmod{65}$ 
9 :  $3 - 1 \geq 0 \rightarrow x \equiv 61^2 \pmod{65} = 16$ 
10 :  $16 \not\equiv -1 \pmod{65}$ 
11 :  $2 - 1 \geq 0 \rightarrow x \equiv 16^2 \pmod{65} = 61$ 
12 :  $61 \not\equiv -1 \pmod{65}$ 
13 :  $1 - 1 \geq 0 \rightarrow x \equiv 61^2 \pmod{65} = 16$ 
14 :  $16 \not\equiv -1 \pmod{65}$ 
15 :  $0 - 1 \not\geq 0 \rightarrow n$  composto

```

Avremo che $a = 2$ e' un *MR-Testimone* di $n = 65$.

Verifichiamo ora per $a = 5$, $MCD(5, 65) > 1$.

1 : $x \equiv 5^1 \pmod{65} = 5$
2 : $5 \not\equiv \pm 1$
3 : $6 - 1 \geq 0 \rightarrow x \equiv 5^2 \pmod{65} = 25$
4 : $25 \not\equiv -1 \pmod{65}$
5 : $5 - 1 \geq 0 \rightarrow x \equiv 25^2 \pmod{65} = 40$
6 : $40 \not\equiv -1 \pmod{65}$
7 : $4 - 1 \geq 0 \rightarrow x \equiv 40^2 \pmod{65} = 40$

...

11 : $0 - 1 \not\geq 0 \rightarrow n$ composto

Quindi troviamo che $a = 5$ e' un *MR-Testimone Banale* di $n = 65$.

Mostro adesso un'implementazione dell'Algoritmo `MCPrimalityTest(65)` per $a = 2, \dots, 63$.

```
[2]: # Imports
import IPython
import warnings

import random
```

```
[ ]: # MCPrimalityTest(n)
# Input: n >=3 dispari
# Output: "n composto" o "probabilmente primo"

def MCPrimalityTest(n, a):
    if n < 3 or n % 2 == 0:
        return "NON VALID INPUT"
    s = 0
    q = n - 1
    while q % 2 == 0:
        s += 1
        q = q // 2
    x = pow(a, q, n)
    if x == 1 or x == -1:
        return "probably prime"
    while s - 1 > 0:
        x = pow(x, 2, n)
        if x == n - 1:
            return "probably prime"
        s -= 1
    return "composite"
```

```

# Output
n = 65
liars = []
witnesses = []

for a in range(2, n - 1):
    res = MCPrimalityTest(n, a)

    if res == "probably prime":
        liars.append(a)
    else:
        witnesses.append(a)

print(f"MR-Liars: {liars}")
print(f"MR-Witnesses: {witnesses}")

```

MR-Liars: [8, 18, 47, 57]

MR-Witnesses: [2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 48, 49, 50, 51, 52, 53, 54, 55, 56, 58, 59, 60, 61, 62, 63]

Notiamo dall'output dell'algoritmo come esso corrisponda esattamente ad $H_{65} = \{1, 8, 14, 18, 47, 51, 57, 64\}$ meno i due *MR-Testimoni* scovati (e quelli banali). Questo insieme e' **sottogruppo proprio** di $\mathbb{Z}_{65}^*$ (ossia $|H_{65}| < |\mathbb{Z}_{65}^*|$), infatti esiste una base $\bar{a}$ per cui $\bar{a}^{64} \not\equiv 1 \pmod{65}$ (un *MR-Testimone*). e' anche verificato il **Teorema di Lagrange** ($|\mathbb{Z}_{65}^*| = (5-1)(13-1) = 48 \rightarrow |H_{65}| \in \{d \in \mathbb{N} \mid 48 \pmod{d} = 0\}$), il quale afferma che al massimo la meta' delle basi apparterranno a H_{65} e dunque almeno meta' delle basi saranno *MR-Testimoni*.

La complessita' dell'algoritmo, se $\log n$ sono i bit necessari per codificare un numero n , il costo di k ripetizioni dell'algoritmo e' pari a $O(k(\log n)^3)$.

Baldini Filippo - 6393212